

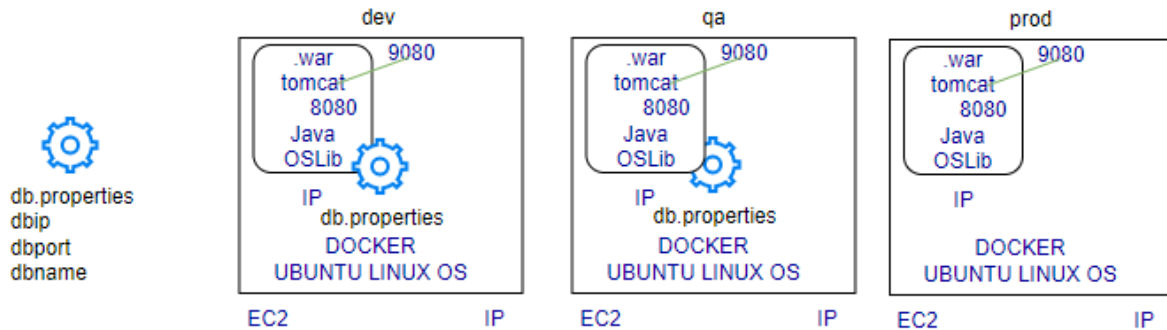
External Configuration (config-server)

container package (Image): Is package that contains everything need to run a application/software.

Environment specific config data can't be packaged in container image.

environment==group of servers where we run our app

In realtime always db servers are different for each environment



Environment specific configuration data needs to be injected into container when we run the container from image at respective environment.

How to inject the config data into cont at runtime?

- configmap- plain config data
- secret: sensitive data

UC: My application requirement is to connect to database, will connect to database through **connection URL**, can we keep database URL inside image?

No: Image is prepared during development and contains connection string of dev database, can't be build image with dev database URL, if we ship the same image in production will not work, we cannot put database details inside image which requires **environment level configuration** not the application level, our requirement is to deploy application in different environment.

In this case there is still something which is specific to the environment, means **environment level configuration can't be put inside image**, so that we need some configuration at environment level which can't be keep inside image.

Why Configuring Containerized Applications?

- Container images are built to be “portable”.
- Decouple the configuration from pods and components.
- Store configuration data as key-value pairs

Env Variable in Kubernetes

Containers expect configuration from

- Configuration Files: (configMap and Secrets)
- Command Line Arguments
- Environment Variables: (plain key-value pair)

UC: There are multiple environments in my organization, how can we pass URL dynamically. We Can provide data dynamically to container through Volumes or through the env variable, If we want to send some data through the files then go for volumes, if we want to send some data as environment variable then we use ENV.

ConfigMap

- Is a way of storing some plain configuration data in the kube cluster itself.
- Once the data is stored in the cluster, we can inject the data into container at run time.
 - As volume data

If we have one or two env variables, we can define in Pod definition file, if you have lots of Pod definition files it will become difficult to manage the env data to store in this definition files we have to take this information out of the Pod definition and manage centrally using configmap.

A ConfigMap is an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMap as environment variables, command-line arguments, or as configuration files in a volume.

A ConfigMap allows you to decouple environment-specific configuration from your container images, so that your applications are easily portable.

Is kind of Storage, when you create a ConfigMap it creates a data and that data stored in the clustered database (etcd) database, and managed by apiServer.

Small information

Sharing Information among multiple Pods (cluster level)

Read Only

Limitation: ConfigMap does not provide secrecy or encryption.

If the data you want to store are confidential, use a Secret rather than a ConfigMap, or use additional (third party) tools to keep your data private.

There are two steps in configMap

- 1) Create ConfigMap
- 2) Inject into Pod

Creating ConfigMap

Imperative approach

kubectl create configmap <map-name> data-source, data-source can be

- Directories
- Files
- Literal Values

```
kubectl create configmap <map-name> <data-source>
```

Declarative approach

Create ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

Declarative ▶ kubectl create -f

config-map.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```

1 Create ConfigMap

▶ kubectl create -f config-map.yaml

app-config

```
APP_COLOR: blue
APP_MODE: prod
```

mysql-config

```
port: 3306
max_allowed_packet: 128M
```

redis-config

```
port: 6379
rdb-compression: yes
```

KodeKloud.com

kubectl get configmaps

kubectl describe configmaps

How to access ConfigMap Inside the Pod?

There are two ways

- Volume - files

- envVariables - variable

ConfigMap allow you to decouple configuration artifacts from image, content to keep containerized application portable.

Portable image: same image can be run on any environment

UC: create a db. property, in order to provide. properties values through volumes
Whenever we create a configMap save data in etcd database.

kubectl create cm <dbconfigmap> --from-file='db. properties'

values retrieved from etcd database available to the pod either using volumes or using env variable.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: dbconf
  namespace: dev
data:
  DBHOST: "10.5.6.7"
  DBPORT: "1345"
  DBNAME: "devdb"
  DBURL: "jdbc:then@10.5.6.7:1345/devdb"

kind: ConfigMap
apiVersion: v1
metadata:
  name: dbconf
  namespace: qa
data:
  DBHOST: "4.21.3.5"
  DBPORT: "1345"
  DBNAME: "qadb"
  DBURL: "jdbc:then@4.21.3.5:1345/qadb"
```

Secrets:

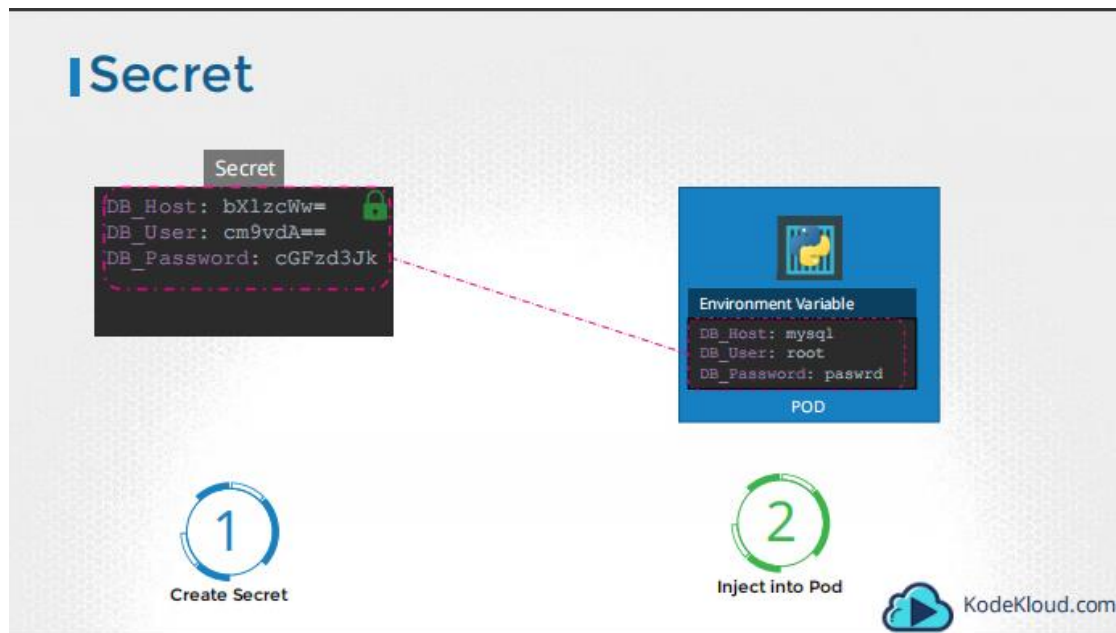
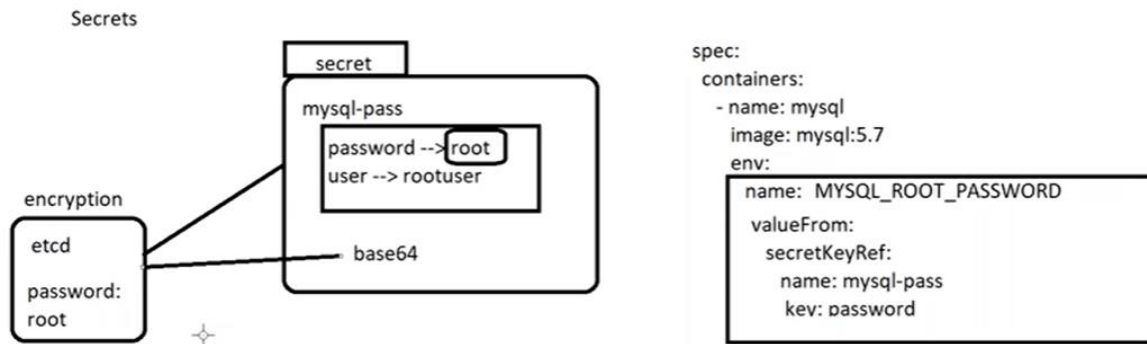
- Is a way of storing some sensitive configuration data in the kube cluster itself.
- Once the data is stored in the cluster, we can inject the data into container at run time.
 - As volume data

Kubernetes Object to handle small amount of sensitive data. (password, tokens or keys).

The main purpose of secrets is to reduce the risk of the accidental exposing of sensitive data.

- Created outside Of Pods.
- Stored inside ETCD database on Kubernetes Master
- Not more than 1 MB
- Used in two ways -Volumes or Env Variables

Sent only to the target node, not broadcast to all the nodes.



There are two steps in Secrets

- Create Secret
- Inject into Pod

Creating Secrets

Imperative approach

Don't create a config file to create a secret object, instead we are going to run imperative command

```
kubectl create secret [TYPE] [NAME] [DATA]
```

generic

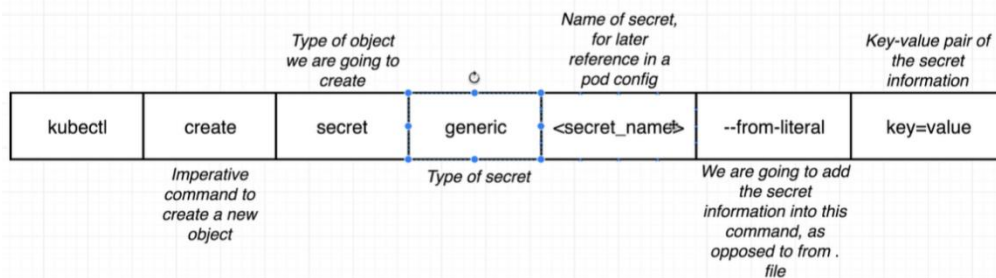
- File
- Directory
- Literal Value

docker-registry

tls

- Path to dir/file: `--from-file`
- Key-Value pair : `--from-literal`

Creating a Secret



Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

Imperative

```
kubectl create secret generic
<secret-name> --from-literal=<key>=<value>
```

```
kubectl create secret generic \
app-secret --from-literal=DB_Host=mysql \
--from-literal=DB_User=root
--from-literal=DB_Password=paswrd
```

```
kubectl create secret generic
<secret-name> --from-file=<path-to-file>
```

```
kubectl create secret generic \
app-secret --from-file=app_secret.properties
```



Create Secret

KodeKloud.com

Using Declarative approach

Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswr
```

Declarative

```
kubectl create -f
```

1

Create Secret

```
secret-data.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: bXlzcWw=
  DB_User: cm9vdA==
  DB_Password: cGFzd3Jk
```

```
kubectl create -f secret-data.yaml
```

KodeKloud.com

Secrets: Stores sensitive information, first encrypt the data using base 64 or other cryptography

echo admin | base 64

kubectl create secret generic <db secret> --from-file='db. properties'

How to decode?

echo ec9vda== | base64 -d

How to validate the environment variables inside the Pod?

```
root@cloudshell:~$ kubectl exec -it cm-env-var-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=cm-env-var-pod
TERM=xterm
database_uri=mongodb://localhost:27017
database=mongodb
KUBERNETES_PORT_443_TCP=tcp://10.8.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.8.0.1
KUBERNETES_SERVICE_HOST=10.8.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
```